Thread-Safe Singleton Pattern - Modern C++ Implementation

Theory, Edge Cases & Code Examples

- ✓ Singleton Pattern
- ✓ Classic Singleton (NOT Thread-Safe)
- ✓ Meyer's Singleton (C++11 Thread-Safe)
- ✓ Double-Checked Locking Pattern (DCLP)
- ✓ Destruction Order (Static Deinitialization Fiasco)

→ *Swipe to learn* →

Thread-Safe Singleton Pattern - Modern C++ Implementation

Theory

📌 1. Singleton Pattern

Goal: Ensure only one instance of a class exists globally.

Use cases:
- Logger
- Configuration manager
- Database connection pool

Thread-Safe Singleton Pattern - Modern C++ Implementation

Theory

📌 2. Classic Singleton (NOT Thread-Safe)

```
class Singleton {
private:
    static Singleton* instance_;

    Singleton() {}  // Private constructor

public:
    static Singleton* getInstance() {
        if (!instance_) {
            instance_ = new Singleton();  // Race condition!
        }
        return instance_;
    }
};

Singleton* Singleton::instance_ = nullptr;
```

Problem: Two threads can create two instances.

Thread-Safe Singleton Pattern - Modern C++ Implementation

Theory

📌 3. Meyer's Singleton (C++11 Thread-Safe)

```
class Singleton {
private:
    Singleton() {}

public:
    static Singleton& getInstance() {
        static Singleton instance;  // Thread-safe initialization (C++11)
        return instance;
    }

    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;
};
```

Key: C++11 guarantees static local variables initialized once, thread-safely.

Thread-Safe Singleton Pattern - Modern C++ Implementation

Theory

📌 4. Double-Checked Locking Pattern (DCLP)

```
class Singleton {
private:
    static std::atomic<Singleton*> instance_;
    static std::mutex mutex_;

    Singleton() {}

public:
    static Singleton* getInstance() {
        Singleton* tmp = instance_.load(std::memory_order_acquire);

        if (!tmp) {  // First check (no lock)
            std::lock_guard<std::mutex> lock(mutex_);
            tmp = instance_.load(std::memory_order_relaxed);

            if (!tmp) {  // Second check (with lock)
                tmp = new Singleton();
                instance_.store(tmp, std::memory_order_release);
            }
        }

        return tmp;
    }
};

std::atomic<Singleton*> Singleton::instance_{nullptr};
std::mutex Singleton::mutex_;
```

Optimization: Avoids lock on every call (only locks during initialization).

Complete Implementations

Thread-Safe Singleton Pattern - Modern C++ Implementation

Edge Cases

📌 1. Destruction Order (Static Deinitialization Fiasco)

```
class Singleton1 {
public:
    static Singleton1& get() {
        static Singleton1 instance;
        return instance;
    }

    ~Singleton1() {
        Singleton2::get().doSomething();  // ← May crash if Singleton2 already destroyed
    }
};
```

Solution: Avoid dependencies between singletons' destructors.

Thread-Safe Singleton Pattern - Modern C++ Implementation

Edge Cases

📌 2. Lazy vs Eager Initialization

Lazy (Meyer's):

```
static Singleton& get() {
    static Singleton instance;  // Created on first call
    return instance;
}
```

Eager:

```
class Singleton {
    static Singleton instance;  // Created at program start
};

Singleton Singleton::instance;
```

Thread-Safe Singleton Pattern - Modern C++ Implementation

Code Examples

📌 1. Thread-Safe Logger (1/2)

This example demonstrates a production-ready thread-safe file logger using Meyer's Singleton pattern with mutex protection for concurrent writes.

What this code does:

- Implements a singleton logger that writes to a single log file (app.log) opened in append mode
- Opens the log file lazily on first access to getInstance() (not at program startup)
- Protects each log write with a mutex to prevent interleaved output from multiple threads
- Flushes the file stream after each write to ensure messages are persisted immediately
- The singleton instance and its file handle are automatically destroyed at program exit

Key concepts demonstrated:

- Meyer's Singleton ensures exactly one Logger instance exists across all threads
- C++11 guarantees thread-safe initialization of static local variables (no race conditions)
- std::mutex prevents data races when multiple threads write to the file simultaneously
- std::lock_guard provides RAII-based lock management (automatic unlock on scope exit)
- Opening file in append mode (std::ios::app) prevents overwriting previous runs' logs

Real-world applications:

- Application logging in multi-threaded servers where all threads need to log events
- Error reporting systems that must work even during stack unwinding
- Diagnostic logging in libraries that don't know about caller's threading model
- Centralized audit trails in financial or security-critical applications

Why this matters:

- Without mutex protection, log lines from different threads would be garbled and intermixed
- Without singleton pattern, multiple log files might be created or writes might conflict
- Immediate flushing trades performance for reliability (logs survive crashes)
- Global access via getInstance() works from any context without dependency injection

Performance implications:

- Mutex contention can become a bottleneck if logging is extremely frequent
- Flushing after each write is slow but ensures crash-resistant logging
- Consider buffering or async logging for high-throughput applications
- First call to getInstance() has slight overhead due to static initialization

---

Thread-Safe Singleton Pattern - Modern C++ Implementation

Code Examples

📌 1. Thread-Safe Logger (2/2)

```
#include <fstream>
#include <mutex>

class FileLogger {
private:
    std::ofstream file_;
    std::mutex mutex_;

    FileLogger() : file_("app.log", std::ios::app) {}

public:
    static FileLogger& getInstance() {
        static FileLogger instance;
        return instance;
    }

    void log(const std::string& message) {
        std::lock_guard<std::mutex> lock(mutex_);
        file_ << message << '\n';
        file_.flush();
    }
};

// Usage:
FileLogger::getInstance().log("Application started");
```

What You Learned

Key Takeaways

- ✓ Singleton Pattern
- ✓ Classic Singleton (NOT Thread-Safe)
- ✓ Meyer's Singleton (C++11 Thread-Safe)
- ✓ Double-Checked Locking Pattern (DCLP)

→ *Swipe to learn* →